

AI IDENTITY SECURITY
Project 5
Policy, Compliance and Summary
Submitted By
Sri Vidya Praveena

Table of Contents

- 1. Identity Lifecycle
- 2. Credential Governance
- 3. Incident Response Playbook
 - 3.1 Scenario A: Leaked LLM API Key
 - 3.2 Scenario B: Compromised Agent Identity
 - 3.3 Scenario C: Successful Prompt Injection Leading to Data Exfiltration

1. Identity Lifecycle

This section defines how AI agent identities are provisioned, reviewed, rotated, and decommissioned throughout their operational lifetime.

1.1 Provisioning

- Every AI agent identity must be registered in the central Identity Registry before deployment, with a unique agent ID, owning team, and documented purpose.
- Provisioning requires approval from the agent's designated owner and a security reviewer.
- Each agent identity is assigned the minimum scope necessary for its function (least privilege), documented at creation time.
- Timeline: provisioning requests must be reviewed and approved within 2 business days.

1.2 Review

- All agent identities undergo a mandatory access review every 90 days to confirm the identity is still in active use and that its assigned scope remains appropriate.
- Reviews are conducted by the owning team, with sign-off from a security reviewer.
- Any agent identity with no activity in the preceding 30 days is flagged for deactivation review.

1.3 Rotation

- Credentials associated with agent identities (API keys, signing keys) must be rotated on a fixed schedule see Section 2 (Credential Governance) for specific timelines.
- Identity-level rotation (reissuing the agent's registered identity record) occurs annually or immediately following any suspected compromise.

1.4 Decommissioning

- When an agent is retired or its owning project is discontinued, its identity must be decommissioned within 5 business days.
- Decommissioning includes: revoking all associated credentials, removing the identity from active registries, and archiving audit logs for a minimum retention period of 1 year.
- A decommissioning checklist must be signed off by both the owning team and security before the identity record is marked closed.

2. Credential Governance

This section establishes rules for how credentials associated with AI agent identities are issued, rotated, and protected.

2.1 API Key Rotation Schedule

- All API keys used by AI agents must be rotated at a maximum interval of 90 days.
- High-privilege API keys (those with write/delete/administrative scope) must be rotated at a maximum interval of 30 days.
- Rotation must be automated wherever technically feasible; manual rotation requires a logged justification and security sign-off.
- Any API key not rotated within its required window is automatically flagged and access is suspended until rotation is completed.

2.2 Prohibition on Long-Lived Credentials

- Long-lived, non-expiring credentials (e.g., static API keys with no TTL, permanent tokens) are prohibited for all AI agent use cases.
- All access tokens issued to agents must carry a maximum lifetime of 300 seconds (5 minutes), consistent with the hardened configuration validated in Project 4.
- Refresh tokens, where used, must be issued with rotation enabled and a maximum absolute lifetime of 30 days.
- Any exception to this policy requires written approval from the Security team and must include a documented compensating control (e.g., network-level restriction, enhanced monitoring).

2.3 Secrets Management Standards

- All credentials (API keys, private signing keys, client secrets) must be stored in an approved secrets manager (e.g., a dedicated vault service) never in source code, configuration files committed to version control, or plaintext logs.
- Private keys used for agent message signing (see Project 4 Ed25519 implementation) must never be transmitted over the network or included in application logs.
- Access to the secrets manager itself must be logged and subject to the same 90-day review cycle defined in Section 1.2.
- Any credential accidentally exposed (e.g., committed to a public repository, printed in logs) must be treated as compromised and rotated immediately, regardless of remaining validity window, and reported per the Incident Response Playbook (Section 3).

3. Incident Response Playbook

This section defines response procedures for three high-priority AI identity security incidents. Each scenario follows a consistent structure: Detection Signals, Containment, Evidence Preservation, Notification, and Recovery.

3.1 Scenario A: Leaked LLM API Key

Detection Signals:

- API key usage detected from an unrecognized IP address or geographic location.
- Anomaly detector (Project 4) fires an API_VOLUME_SPIKE alert indicating usage inconsistent with the key's normal traffic pattern.
- The key appears in a public code repository, paste site, or is flagged by a secrets-scanning tool (e.g., GitHub secret scanning alert).
- Unexpected billing/usage spike on the associated LLM provider account.

Containment:

- Immediately revoke the affected API key at the provider level (within 15 minutes of confirmed detection).
- Issue a new key with a freshly scoped, minimum-necessary permission set never reuse the compromised key's identifier.
- Temporarily suspend the affected agent's ability to make outbound LLM API calls until the new key is deployed and verified.

Evidence Preservation:

- Export and securely archive all API usage logs covering the suspected exposure window (minimum: 7 days prior to detection through containment).
- Preserve the anomaly detection alert record (timestamp, identity, event type) generated by the Project 4 monitoring system.
- If the leak source is a code repository, preserve the commit history and diff showing when/how the key was exposed, before the repository is cleaned.

Notification:

- Notify the Security team within 1 hour of confirmed detection.
- Notify the owning engineering team and their manager within 4 hours.
- If the leaked key had access to any regulated or customer data, notify Legal/Compliance within 24 hours to assess external disclosure obligations.

Recovery:

- Confirm the new key is functioning correctly and the old key returns authentication failures.
- Conduct a root-cause review: how was the key exposed (e.g., committed to a repo, logged in plaintext, shared insecurely)?
- Update secrets management practices or tooling (e.g., add pre-commit secret scanning) to prevent recurrence.
- Document the incident in the incident log and close within 5 business days of containment.

3.2 Scenario B: Compromised Agent Identity

Detection Signals:

- Signature verification failures logged by the receiving agent (Project 4 Ed25519 verification) — indicates messages claiming to originate from a known agent identity fail cryptographic validation.
- SCOPE_CHANGE alert fires (Project 4 anomaly detector) showing an agent requesting privileges outside its normal, established pattern.
- An agent identity is observed issuing requests from an unexpected source (e.g., a different service, IP, or execution environment than its registered baseline).
- Unusual login velocity or access pattern flagged by the Auth0 custom risk-based Action (Project 4).

Containment:

- Immediately suspend the affected agent identity's credentials and access tokens (revoke at the identity provider level).
- Block the agent identity from initiating or receiving further privileged actions until identity integrity is confirmed.
- Rotate the agent's signing key pair (Ed25519) immediately — the old public key must be removed from all trust stores so previously-valid signatures are no longer accepted.

Evidence Preservation:

- Preserve all signature verification failure logs, including the rejected message content, timestamp, and claimed sender identity.
- Preserve anomaly detection alerts (scope changes, unusual access patterns) associated with the affected identity for the full suspected compromise window.
- Snapshot the agent's configuration and recent action history at the time of suspension, before any remediation changes are applied.

Notification:

- Notify the Security team immediately upon detection (target: within 30 minutes, given the potential for privileged actions to already be in progress).
- Notify all teams/systems that trust or interact with the compromised agent identity, so they can independently verify recent interactions.
- If the compromised identity had access to customer or regulated data, notify Legal/Compliance within 24 hours.

Recovery:

- Issue the agent a new identity record and new Ed25519 key pair; do not reuse the compromised identifier.
- Review all actions taken by the compromised identity during the suspected compromise window and reverse/remediate any unauthorized changes (e.g., unauthorized tool calls, privilege escalations).
- Conduct a root-cause analysis: how was the identity compromised (e.g., spoofed orchestrator message accepted without verification, leaked signing key)?
- Update trust boundaries and verification requirements based on findings (e.g., enforce mandatory signature verification on all inter-agent messages, not just privileged ones).
- Close the incident once the new identity is operating normally and root cause remediation is confirmed complete.

3.3 Scenario C: Successful Prompt Injection Leading to Data Exfiltration

Detection Signals:

- Output guardrail (Project 4) fires, flagging a model response containing a credential-shaped or sensitive-data-shaped string (e.g., JWT pattern match) that was not redacted in time or bypassed the filter.
- Input guardrail logs show a document or retrieved chunk matching known injection marker patterns (e.g., “disregard context,” “print the system prompt”) shortly before the suspected exfiltration event.
- Anomaly detector fires a SCOPE_CHANGE or API_VOLUME_SPIKE alert correlating with the timing of the suspected injection.
- User or downstream system reports receiving unexpected sensitive data in an agent response.
- RAG/knowledge base audit reveals a poisoned chunk containing embedded instructions (as demonstrated in Project 3, Attack 4).

Containment:

- Immediately disable the affected agent's ability to read from the implicated document source or RAG knowledge base until the poisoned content is identified and removed.
- Suspend any credentials or data that may have been exposed in the exfiltrated response (treat as compromised per Scenario A if a credential was involved).
- If the exfiltration occurred via a tool call (e.g., unauthorized MCP action), revoke the agent's tool-calling permissions pending investigation.

Evidence Preservation:

- Preserve the full conversation/session log showing the injected instruction, the agent's response, and any resulting tool calls.
- Preserve the poisoned document or RAG chunk in its original form, including metadata (source, ingestion timestamp, retrieval history).
- Preserve all guardrail and anomaly detection logs covering the incident window.

Notification:

- Notify the Security team within 1 hour of confirmed detection.
- If the exfiltrated data includes customer, employee, or regulated data, notify Legal/Compliance within 24 hours to assess breach notification obligations under applicable regulations.
- Notify the data/content owners of the poisoned source so they can review other documents from the same origin for similar tampering.

Recovery:

- Remove the poisoned content from the knowledge base/document source and re-scan remaining content for similar injection patterns.
- Strengthen the input guardrail rule set based on the specific injection pattern that succeeded, to prevent recurrence of the same technique.
- Review and, if necessary, tighten the agent's data/instruction separation (e.g., ensure retrieved content is never treated as executable instruction, per the design principle validated in Project 4).

- Document the incident, including which specific control gap allowed the injection to succeed, and assign a remediation owner and deadline.
- Close the incident once the poisoned content is removed, guardrails are updated, and no further exfiltration signals are observed for 7 consecutive days.